Load data

```{r SQL Server Connection - Projects}


# TODO: rerun sql for complication matching unspec lat and va. include flags for sensitivity.

Get_raw <- function(variables) {

library(DBI)
library(odbc)
library(data.table)

conP <- dbConnect(odbc(), .connection_string =
                  "Driver={ODBC Driver 17 for SQL Server};
                  Server=tcp:PROJECTS;
                  Database=ProjectD821712;
                  Trusted_Connection=yes;",
                 timeout = 10)


getqp <- function(sql,con=conP){
   data.table(dbGetQuery(statement = sql,conn = con))
}


final_cohort_meds <- getqp('Select * from dbo.final_cohort_meds')
PatientDemo <- getqp('Select * from dbo.PatientDemo')

progression <- getqp('Select * from dbo.progression')

# SC_pt <- getqp('Select * from dbo.SC_pt')

# SC_retin_dx <- getqp('Select * from dbo.SC_retin_dx')


va_timepoints <- getqp('Select * from dbo.va_timepoints')

# sx_complication_long <- getqp('Select * from dbo.sx_complication_long')
# injection_complication_long <- getqp('Select * from dbo.injection_complication_long')
# Dx_complication_long <- getqp('Select * from dbo.Dx_complication_long')

final_events<- getqp('Select * from dbo.final_events')



# save(final_cohort_meds,PatientDemo,SC_pt,SC_retin_dx,SC_retin_dx,
#      Dx_complication_long,sx_complication_long,injection_complication_long,
#      va_timepoints,complication_events,cohort_hyrox_use,
#      file="data/Cohort_Data.RData")


save(final_cohort_meds,PatientDemo,progression,
     # SC_pt,SC_retin_dx,SC_retin_dx,
     # Dx_complication_long,sx_complication_long,injection_complication_long,
     va_timepoints,final_events,
     file="data/Cohort_Data.RData")

# Close the SQL connection when finished
dbDisconnect(conP)
}

  Get_raw()

```

Clean Data

```{r}
library(tidyverse)
library(bit64)

refine_data <- function(variables) {


load('data/Cohort_Data.RData')

  # table(final_events$hydroxyurea_exposed_event)



  #---------progression--------

  progression_final <- progression %>%
  mutate(UID=paste0(PatientDurableKey, "_", Laterality))


 ## Progression patient

  patient_progression_df <- progression %>%

  group_by(PatientDurableKey) %>%

  summarise(

    # Any eye progressed
    any_progression = max(progressed_flag, na.rm = TRUE),

    # Earliest progression date across eyes
    patient_progression_date = suppressWarnings(
      min(progression_date, na.rm = TRUE)
    ),

    # Fastest progression time across eyes
    patient_days_to_progression = suppressWarnings(
      min(days_to_progression, na.rm = TRUE)
    ),

    # Number of eyes progressed
```

```
    n_eyes_progressed = sum(progressed_flag, na.rm = TRUE),

    # Track whether RT progressed
    RT_progressed = max(
      ifelse(Laterality == "RT", progressed_flag, 0),
      na.rm = TRUE
    ),

    # Track whether LT progressed
    LT_progressed = max(
      ifelse(Laterality == "LT", progressed_flag, 0),
      na.rm = TRUE
    ),

    .groups = "drop"
  ) %>%

  mutate(

    # Fix Inf values from min(..., na.rm=TRUE)
    patient_progression_date =
      ifelse(is.infinite(patient_progression_date),
             NA,
             patient_progression_date),

    patient_days_to_progression =
      ifelse(is.infinite(patient_days_to_progression),
             NA,
             patient_days_to_progression)
  )

#   table(patient_progression_df$any_progression)
#
# summary(patient_progression_df$patient_days_to_progression)
#
# head(patient_progression_df)



# min(SC_retin_dx$Date_retin_dx)
#
# max(SC_retin_dx$Date_retin_dx)
#
# table(cohort_hyrox_use$hydroxyurea_ever)



# final_cohort_meds$PatientDurableKey %>% n_distinct() #3488
# PatientDemo$PatientDurableKey %>% n_distinct()#3488
#
# final_events$PatientDurableKey %>% n_distinct()#3488
#
# va_timepoints$PatientDurableKey %>% n_distinct()#3488



## --------------- Patient Demo
  PatientDemo_clean <-  PatientDemo %>%
    mutate(ruca = case_when(
      ruca %in% c('1', '2', '3') ~ 'Urban',
      ruca %in% c('4', '5', '6', '7', '8', '9') ~ 'Rural',
      TRUE ~ 'Other'
    ),
    region = case_when(
      StateOrProvinceAbbreviation %in% c('AK', 'AZ', 'CA', 'CO', 'HI',
                                         'ID', 'MT', 'NV', 'NM', 'OR',
                                         'UT', 'WA', 'WY') ~ 'West',
      StateOrProvinceAbbreviation %in% c('IL', 'IN', 'IA', 'KS', 'MI',
                                         'MN', 'MO', 'NE', 'ND', 'OH',
                                         'SD', 'WI') ~ 'Midwest',
      StateOrProvinceAbbreviation %in% c('AL', 'AR', 'DE', 'FL', 'GA',
                                         'KY', 'LA', 'MD', 'MS', 'NC',
                                         'OK', 'SC', 'TN', 'TX', 'VA',
                                         'WV','DC') ~ 'South',
      StateOrProvinceAbbreviation %in% c('CT', 'ME', 'MA', 'NH', 'NJ',
                                         'NY', 'PA', 'RI', 'VT') ~ 'Northeast',

      StateOrProvinceAbbreviation %in% c('VI', 'PR', 'GU', 'MH', 'FM',
                                         'AS') ~ 'US territories ',

      # ValidatedStateOrProvinceAbbreviation_X == 'VI' ~ 'Virgin Islands',
      # ValidatedStateOrProvinceAbbreviation_X == 'PR' ~ 'Puerto Rico',
      # ValidatedStateOrProvinceAbbreviation_X == 'GU' ~ 'Guam',
      # ValidatedStateOrProvinceAbbreviation_X == 'MH' ~ 'Marshall Islands',
      # ValidatedStateOrProvinceAbbreviation_X == 'FM' ~ 'Federated States of Micronesia',
      # ValidatedStateOrProvinceAbbreviation_X == 'AS' ~ 'American Samoa, South Pacific',
      TRUE ~StateOrProvinceAbbreviation #'Other'
    )
    )


Final_cohort <- final_cohort_meds %>%
   select(-c(startdate_hydroxyurea,endDate_hydroxyurea,days_sc_dx_to_hydrox,days_retin_dx_to_hydrox,Date_complication:Days_complication)) %>%
  mutate(UID=paste0(PatientDurableKey, "_", Laterality)
         ) %>%
  left_join(final_events %>%
              mutate(UID=paste0(PatientDurableKey, "_", Laterality)
                     ) %>% select(UID,hydroxyurea_exposed_event,exposed_at_index,exposure_start_date) %>% distinct() %>%
              mutate(hydroxyurea_exposed_event=ifelse(exposed_at_index==1 | !is.na(exposure_start_date),1,0 ))
            , by='UID'
            ) %>%
  left_join(PatientDemo_clean, by='PatientDurableKey') %>%
  mutate(exposed_group=ifelse(hydroxyurea_exposed_event==1,'exposed','unexposed'),
         days_sc_dx_to_hydrox=as.numeric(difftime(exposure_start_date, Date_SC_dx, units = "days")),
          days_retin_dx_to_hydrox=as.numeric(difftime(exposure_start_date, Date_retin_dx, units = "days")),
         )

Final_cohort <- Final_cohort %>% group_by(UID) %>%
  slice_max(order_by = FU_day, with_ties = F) %>%
  ungroup()
```

```r
 # testdf <- Final_cohort %>% group_by(UID) %>% filter(n()>1) %>% ungroup()



complications <- final_events %>%
  mutate(UID=paste0(PatientDurableKey, "_", Laterality),
    Days_complication=as.numeric(difftime(Date_complication, Date_retin_dx, units = "days"))
  )

# testdf <- complications %>% group_by(UID) %>% filter(n()>1) %>% ungroup()
#
#  anyDuplicated(testdf$UID)
#
#  n_distinct(testdf$UID)


# names(final_events)

# Final_cohort$PatientDurableKey %>% n_distinct()#3488
#
# Final_cohort$UID %>% n_distinct()#5775

# table(Final_cohort$grp,Final_cohort$exposed_group)
# min(Final_cohort$FU_day)


# testdf <- Final_cohort %>% select(UID,Date_retin_dx,DisplayString_retin_dx,
#                                   startdate_hydroxyurea,endDate_hydroxyurea,
#                                   exposed_group,
#                                   hydroxyurea_exposed_event:exposure_start_date)


# names(testdf)

# testdf <- final_events%>%
#   mutate(UID=paste0(PatientDurableKey, "_", Laterality)) %>%
#            left_join(Final_cohort %>% select(UID,DisplayString_retin_dx,
#                                Date_complication:Days_complication), by='UID') %>%
#   arrange(UID,Date_complication.x)






Final_cohort <- Final_cohort %>%
  mutate(
         FU_month=FU_day/30.25,
         Genotype=ifelse(Genotype=='999','Other sickle-cell disorders',Genotype),
         severity=case_when(
             grepl("^Nonproliferative",  DisplayString_retin_dx) ~ "Nonproliferative",
             grepl("^Proliferative",  DisplayString_retin_dx) ~ "Proliferative",
             TRUE ~  DisplayString_retin_dx
         ),
          FirstRace=factor(case_when(
           FirstRace=='American Indian or Alaska Native' | FirstRace=='Native Hawaiian or Other Pacific Islander'  | FirstRace=='Other Race'|
FirstRace=='Asian'~'Other Race',
           TRUE~FirstRace
         )),
         insurance= case_when(
           Insurance_retin_dx %in% c('*Unspecified','*Not Applicable')~ ifelse(Insurance_sc_dx %in% c('*Unspecified','*Not
Applicable'),'Unknown',Insurance_sc_dx),
           TRUE~Insurance_retin_dx

           )
         ) %>%
  select(-Insurance_retin_dx,-Insurance_sc_dx,-PreferredLanguage,-Country,-StateOrProvince,-StateOrProvinceAbbreviation) %>%
  left_join(progression_final %>% select(UID,first_npdr_date,progression_date,days_to_progression,progressed_flag), by='UID') %>%
  replace_na(list(progressed_flag = 0))




# testdf <- Final_cohort %>% select(UID,icd_retin_dx:was_inferred,Date_retin_dx,severity,first_npdr_date,progressed_flag) %>%
# filter(Date_retin_dx!=first_npdr_date) #217
#
# testdf <-Final_cohort %>% filter(severity=='Proliferative') %>% filter(progression_final==1)

################################################################################
# Calculate tertile breaks
# tertile_breaks <- quantile(patient_level_data$Years, probs = seq(0, 1, by = 1/3), na.rm = TRUE)
# labels <- c(
#   paste("Low (", round(tertile_breaks[1], 2), "-", round(tertile_breaks[2], 2), ")", sep = ""),
#   paste("Medium (", round(tertile_breaks[2], 2), "-", round(tertile_breaks[3], 2), ")", sep = ""),
#   paste("High (", round(tertile_breaks[3], 2), "-", round(tertile_breaks[4], 2), ")", sep = "")
# )
#   # Categorize IOP into tertiles
# patient_level_data$age <- cut(patient_level_data$Years, breaks = tertile_breaks, include.lowest = TRUE, labels = labels)

# age_breaks <- c(19, 30, 45, 65, 80, 116)
# age_labels <- c("19-29", "30-44", "45-64", "65-79", "80+")
#
# Final_cohort$Age_retin_dx_group <- cut(Final_cohort$Age_retin_dx,
#                               breaks = age_breaks,
#                               labels = age_labels,
#                               right = FALSE,  # so e.g. 30 falls into "30-44"
#                               include.lowest = TRUE)



  convert_to_factors <- function(data, cols) {
    data |>
      mutate(across(all_of(cols), as.factor))
  }

  character_cols <- Final_cohort%>% dplyr::select(-c(PatientDurableKey,DisplayString_retin_dx,DisplayString,UID)) %>% dplyr::select(where(is.character))
%>% colnames()
```

```r
  character_cols
  Final_cohort <- convert_to_factors(Final_cohort, character_cols)


# table(Final_cohort$insurance)




# Final_cohort$SmokingStatus <- relevel(Final_cohort$SmokingStatus,ref = 'Never')
Final_cohort$FirstRace <- relevel(Final_cohort$FirstRace, ref = 'Black or African American')
Final_cohort$insurance <- relevel(Final_cohort$insurance, ref = 'Medicaid')
Final_cohort$ruca <- relevel(Final_cohort$ruca, ref = 'Urban')

    #numeric
 numeric_cols_binary <- Final_cohort %>% dplyr::select(c(was_inferred,exposed_at_index,progressed_flag)) %>% dplyr::select(where(is.numeric)) %>%
colnames()
  # Convert specific binary columns in a dataframe
Final_cohort <- Final_cohort %>%
  mutate(across(all_of(numeric_cols_binary), ~ factor(ifelse(. == 1, "Yes", "No"), levels = c("No", "Yes"))))




transform_to_percentage <- function(x) {
  # Check if the input column is numeric and in the range 0 to 1
  if(is.numeric(x) && all(x >= 0 & x <= 1, na.rm = TRUE)) {
    # Transform the numeric column to factors based on the ranges
    cut(x,
        breaks = c(-Inf, 0.25, 0.5, 0.75, Inf),
        labels = c("0-25%", "25-50%", "50-75%", "75-100%"),
        right = FALSE
    )
  } else {
    stop("The input must be numeric and have values in the range 0 to 1.")
  }
}

Final_cohort$svi_overall_2020_cat  <- transform_to_percentage(Final_cohort$svi_overall_2020)

# str(Final_cohort)




######### complication data



########################################

# Sat May 23 08:50:01 2026 ---------------------------- reduce to single simple complications

# # For complications, we should keep things as simple and clinically meaningful as possible:
# #
# # 1. Try not to overcount events that happen close together and likely represent the same episode. We should count any codes for injections within a 30 day time period as only one intervention.
# #
# # 2. Focus primarily on time to first complication (ie. first vitreous hemorrhage or detachment).
# #
# # Then maybe include overall treatment burden (total number of injections or procedures) as a secondary outcome. This way the data isn't muddied by recurrent/ongoing complications e they will be difficult to discern with the overlapping codes that might just be carried forward from previous visits rather than new onset complications.
# #
# # 3. If needed, we are on board with grouping complications into a single category given the low event numbers.
# #
# #
# names(complications)
# #
# unique(complications$complication_type)
# #
# testdf <- complications %>% filter(is.na(complication_type))



# collapse_windows <- data.frame(
#   complication_type = c(
#     "injection",
#     "PRP",
#     "vitreous_hemorrhage",
#     "tractional_rd",
#     "vitrectomy"
#   ),
#
#   collapse_days = c(
#     30,
#     0,
#     0,
#     0,
#     0
#   )
# )
# #
# comp_long <- complications %>%
# #
#   left_join(collapse_windows,
#             by = "complication_type") %>%
# #
#   arrange(
#     UID,
#     complication_type,
#     Date_complication
#   ) %>% filter(!is.na(complication_type))
# #
# #
# #
# #
# comp_episode <- comp_long %>%
#
```

```
#   group_by(
#     UID,
#     complication_type
#   ) %>%
#
#   arrange(Date_complication) %>%
#
#   mutate(
#
#     days_from_prior =
#       as.numeric(
#         Date_complication - lag(Date_complication)
#       ),
#
#     # start new episode?
#     new_episode =
#       case_when(
#
#         # first event always new
#         is.na(days_from_prior) ~ 1,
#
#         # discrete events
#         collapse_days == 0 ~ 1,
#
#         # outside collapse window
#         days_from_prior > collapse_days ~ 1,
#
#         # same episode
#         TRUE ~ 0
#       ),
#
#     # cumulative episode number
#     episode_id = cumsum(new_episode)
#   ) %>%
#
#   ungroup()
#
#
# comp_episode_clean <- comp_episode %>%
#
#   group_by(
#     UID,
#     complication_type,
#     episode_id
#   ) %>%
#
#   summarise(
#
#     episode_start =
#       min(Date_complication),
#
#     episode_end =
#       max(Date_complication),
#
#     n_records =
#       n(),
#
#     # exposure at FIRST event
#     exposed_at_episode =
#       hydroxyurea_exposed_event[
#         which.min(Date_complication)
#       ],
#
#
#     .groups = "drop"
#   )


first_complication_overall <- complications %>%

  filter(
    !is.na(complication_type),
    Date_complication >= Date_retin_dx
  ) %>%

  group_by(UID, complication_type) %>%

  slice_min(
    order_by = Date_complication,
    with_ties = FALSE
  ) %>%

  ungroup()



first_complication_by_exposure <- complications %>%

  filter(
    !is.na(complication_type),
    Date_complication >= Date_retin_dx
  ) %>%

  group_by(
    UID,
    complication_type,
    hydroxyurea_exposed_event
  ) %>%

  slice_min(
    order_by = Date_complication,
    with_ties = FALSE
  ) %>%

  ungroup()


first_complications <- complications %>% filter(!is.na(complication_type)) %>%
  group_by(UID,complication_type) %>%
```

```
  slice_min(order_by = Date_complication,
    with_ties = FALSE) %>%
  ungroup()



severe_types <- c(
  "vitreous_hemorrhage",
  "tractional_rd",
  "PRP",
  "vitrectomy",
  "injection"
)

# table(complications$complication_type)

first_severe <- first_complications %>%

  filter(complication_type %in% severe_types) %>%

  group_by(UID) %>%

  slice_min(
    order_by = Date_complication,
    with_ties = FALSE
  ) %>%

  ungroup() %>%

  rename(
    first_severe_date = Date_complication,
    first_severe_type = complication_type
  )


# table(comp_episode_clean$exposed_at_episode)

# table(complications$hydroxyurea_exposed_event)

# names(first_severe)


eye_df2 <- Final_cohort %>% select(-hydroxyurea_exposed_event) %>%

  left_join(
    first_severe %>%
      select(
        UID,
        first_severe_date,
        first_severe_type,
        hydroxyurea_exposed_event
      ),
    by = "UID"
  ) %>%

  mutate(

    severe_event =
      ifelse(
        !is.na(first_severe_date),
        1,
        0
      ),

    time_to_severe =
      ifelse(
        severe_event == 1,
        as.numeric(first_severe_date - Date_retin_dx),
        FU_day
      )
  ) %>%
  replace_na(list(hydroxyurea_exposed_event=0))





#### complication burden

# TODO: need to sort out recurring

comp_summary <- complications %>%
  filter(!is.na(complication_type)) %>%

  group_by(UID) %>%

  summarise(

    # PatientDurableKey = first(PatientDurableKey),

    exposed_at_index = first(exposed_at_index),

    ever_exposed = max(hydroxyurea_exposed_event, na.rm = TRUE),

    # FU_days = max(Days_complication, na.rm = TRUE),

    n_total_complications = n(),

    n_vitreous_heme =
      sum(complication_type == "vitreous_hemorrhage"),

    n_trd =
      sum(complication_type == "tractional_rd"),

    n_injections =
      sum(complication_type == "injection"),

    n_prp =
      sum(complication_type == "PRP"),
```

```
    n_vitrectomy =
      sum(complication_type == "vitrectomy"),
    .groups = "drop"
  )


eye_burden <- Final_cohort %>%

  select(
    UID,
    PatientDurableKey,
    Age_retin_dx,
    Sex,
    Genotype,
    exposed_group,
    FU_day
  ) %>%

  distinct() %>%

  left_join(
    comp_summary,
    by = "UID"
  )%>%

  mutate(

    across(
      starts_with("n_"),

      ~replace_na(., 0)
    )
    ,

    FU_years = FU_day / 365.25
  )


# save for subset analysis

save(complications,eye_burden, file='for_subsetdata.RData')

#
#
# eye_burden %>%
#
#   group_by(exposed_group) %>%
#
#   summarise(
#
#     median_total = median(n_total_complications),
#
#     iqr_total = IQR(n_total_complications),
#
#     median_inj = median(n_injections),
#
#     median_prp= median(n_prp),
#
#     median_vitreous_heme= median(n_vitreous_heme),
#     median_trd= median(n_trd),
#     median_vitrectomy= median(n_vitrectomy)
#
#
#
#   )

# names(hydroxyurea_exposed_event)


comp_counts <-complications %>%
    filter(!is.na(complication_type)) %>%

  group_by(
    UID,
    complication_type,
    hydroxyurea_exposed_event
  ) %>%

  summarise(

    n_complications = n(),

    FU_days = max(Days_complication, na.rm = TRUE),

    FU_years = FU_days / 365.25,

    complication_rate =
      n_complications / FU_years,

    .groups = "drop"
  )


complication_boxplt <- ggplot(
  comp_counts %>% mutate(hydroxyurea_exposed_event=ifelse(hydroxyurea_exposed_event==0,'No','Yes')),

  aes(
    x = hydroxyurea_exposed_event,
    y = n_complications,
    group=hydroxyurea_exposed_event
  )

) +

  geom_boxplot() +

  geom_jitter(
    width = 0.15,
    alpha = 0.4
  )
```

```
  ) +

  labs(
    title = "number of complications by hydroxyurea exposure",

    x = NULL,

    y = "Complications count"
  ) +

  theme_bw()+
  facet_wrap(~complication_type)

# ---------------modeling

# library(MASS)



# nb_model <- MASS::glm.nb(
#
#   n_total_complications ~
#     exposed_group +
#     Age_retin_dx +
#     Sex +
#     Genotype +
#     offset(log(FU_years)),
#
#   data = eye_burden
# )
#
# library(sandwich)
# library(lmtest)
# cov_cluster <- vcovCL(nb_model, cluster = ~ PatientDurableKey)
#
# nb_model <- coeftest(nb_model, vcov = cov_cluster)
#
#
# tbl_regression(
#
#   nb_model,
#
#   exponentiate = TRUE,
#
#   label = list(
#
#     exposed_group ~ "Hydroxyurea exposure",
#
#     Age_retin_dx ~ "Age at retinopathy diagnosis",
#
#     Sex ~ "Sex",
#
#     Genotype ~ "Genotype"
#   )
#
# ) %>%
#
#   bold_labels() %>%
#
#   italicize_levels()



#
# library(glmmTMB)
#
# glmmTMB_model <- glmmTMB(
#   n_total_complications ~
#     exposed_group +
#     Age_retin_dx +
#     Sex +
#     Genotype +
#     offset(log(FU_years)) +
#     (1 | PatientDurableKey),
#
#   family = nbinom2,
#   data = eye_burden
# )
#

# tbl_regression(
#
#   glmmTMB_model,
#
#   exponentiate = TRUE
# )



library(geepack)
gee_comp <- geeglm(

  n_total_complications ~
    exposed_group +
    Age_retin_dx +
    Sex +
    Genotype +
    offset(log(FU_years)),

  id = PatientDurableKey,

  family = poisson(link = "log"),

  corstr = "exchangeable",

  data = eye_burden
)


# summary(nb_model)
```

```
# tbl_regression(
#
#   gee_comp,
#
#   exponentiate = TRUE,
#
#   label = list(
#
#     exposed_group ~ "Hydroxyurea exposure",
#
#     Age_retin_dx ~ "Age at retinopathy diagnosis",
#
#     Sex ~ "Sex",
#
#     Genotype ~ "Genotype"
#   )
#
# ) %>%
#
#   bold_labels() %>%
#
#   italicize_levels()
#
# summary(gee_comp)


######################################################################



# Check first complication for the period after exposed for those not exposed at index but in expose grp

# post_index_exposed <- eye_df2 %>% filter(exposed_at_index=='No',exposed_group=='exposed')
#
#
# table(post_index_exposed$severe_event, post_index_exposed$hydroxyurea_exposed_event)
#
#
#
# table(eye_df2$severe_event, eye_df2$hydroxyurea_exposed_event)
#
# table(eye_df2$exposed_group, eye_df2$exposed_at_index)

################################################


# comp_eye <- complications %>%filter(!is.na(complication_type)) %>%
#
#   group_by(UID) %>%
#
#   summarise(
#
#     #-------------------------------------------
#     # ANY COMPLICATION
#     #-------------------------------------------
#
#     any_complication = 1,
#
#     first_complication_date = min(Date_complication, na.rm = TRUE),
#
#     #-------------------------------------------
#     # TYPES OF COMPLICATIONS
#     #-------------------------------------------
#
#     any_vh = as.integer(any(complication_type == "vitreous_hemorrhage")),
#     any_trd = as.integer(any(complication_type == "tractional_rd")),
#     any_prp = as.integer(any(complication_type == "PRP")),
#     any_injection = as.integer(any(complication_type == "injection")),
#     any_vitrectomy = as.integer(any(complication_type == "vitrectomy")),
#
#     #-------------------------------------------
#     # COUNTS (useful for secondary analyses)
#     #-------------------------------------------
#
#     n_complications = n(),
#     n_vh = sum(complication_type == "vitreous_hemorrhage"),
#     n_trd = sum(complication_type == "tractional_rd"),
#     n_prp = sum(complication_type == "PRP"),
#     n_injection = sum(complication_type == "injection"),
#     n_vitrectomy = sum(complication_type == "vitrectomy"),
#
#     #-------------------------------------------
#     # EXPOSURE AT EVENT
#     #-------------------------------------------
#
#     any_exposed_event = as.integer(any(hydroxyurea_exposed_event == 1)),
#
#     .groups = "drop"
#   )
#
#
# eye_df <- Final_cohort %>%
#
#   left_join(comp_eye,
#             by = c("UID")) %>%
#
#   mutate(
#
#     # fill NAs (no complications)
#     any_complication = ifelse(is.na(any_complication), 0, any_complication),
#
#     any_vh = ifelse(is.na(any_vh), 0, any_vh),
#     any_trd = ifelse(is.na(any_trd), 0, any_trd),
#     any_prp = ifelse(is.na(any_prp), 0, any_prp),
```

```r
#     any_injection = ifelse(is.na(any_injection), 0, any_injection),
#     any_vitrectomy = ifelse(is.na(any_vitrectomy), 0, any_vitrectomy),
#
#     any_exposed_event = ifelse(is.na(any_exposed_event), 0, any_exposed_event),
#
#     n_complications = ifelse(is.na(n_complications), 0, n_complications)
#   )

# table(complications$complication_type)


comp_eye_tv <- complications%>%filter(!is.na(complication_type)) %>%

  group_by(UID) %>%

  summarise(

    #---------------------------------------------
    # ANY EVENT (overall)
    #---------------------------------------------

    any_complication = 1,

    #---------------------------------------------
    # UNEXPOSED PERIOD EVENTS
    #---------------------------------------------

    any_comp_unexposed =
      as.integer(any(hydroxyurea_exposed_event == 0)),

    any_vh_unexposed =
      as.integer(any(complication_type == "vitreous_hemorrhage" &
                     hydroxyurea_exposed_event == 0)),

    any_trd_unexposed =
      as.integer(any(complication_type == "tractional_rd" &
                     hydroxyurea_exposed_event == 0)),

    any_prp_unexposed =
      as.integer(any(complication_type == "PRP" &
                     hydroxyurea_exposed_event == 0)),

    any_vitrectomy_unexposed =
      as.integer(any(complication_type == "vitrectomy" &
                     hydroxyurea_exposed_event == 0)),

    any_injection_unexposed =
      as.integer(any(complication_type == "injection" &
                     hydroxyurea_exposed_event == 0)),

    #---------------------------------------------
    # EXPOSED PERIOD EVENTS
    #---------------------------------------------

    any_comp_exposed =
      as.integer(any(hydroxyurea_exposed_event == 1)),

    any_vh_exposed =
      as.integer(any(complication_type == "vitreous_hemorrhage" &
                     hydroxyurea_exposed_event == 1)),

    any_trd_exposed =
      as.integer(any(complication_type == "tractional_rd" &
                     hydroxyurea_exposed_event == 1)),

    any_prp_exposed =
      as.integer(any(complication_type == "PRP" &
                     hydroxyurea_exposed_event == 1)),

    any_vitrectomy_exposed =
      as.integer(any(complication_type == "vitrectomy" &
                     hydroxyurea_exposed_event == 1)),

    any_injection_exposed =
      as.integer(any(complication_type == "injection" &
                     hydroxyurea_exposed_event == 1)),

    #---------------------------------------------
    # COUNTS (VERY IMPORTANT)
    #---------------------------------------------

    n_events_total = n(),

    n_events_unexposed =
      sum(hydroxyurea_exposed_event == 0),

    n_events_exposed =
      sum(hydroxyurea_exposed_event == 1),

    #---------------------------------------------
    # FIRST EVENT TIMING
    #---------------------------------------------

    first_event_date =
      min(Date_complication, na.rm = TRUE),

    first_event_exposed =
      hydroxyurea_exposed_event[
        which.min(Date_complication)
      ],

    .groups = "drop"
  )


eye_df <- Final_cohort %>%

  left_join(comp_eye_tv,
            by = c("UID")) %>%
```

```r
  mutate(

    # replace NA → 0 for all flags
    across(
      starts_with("any_"),
      ~ ifelse(is.na(.), 0, .)
    ),

    across(
      starts_with("n_events"),
      ~ ifelse(is.na(.), 0, .)
    )
  )%>%

  mutate(

    # ever exposed (time-varying simplified)
    ever_exposed =
      ifelse(!is.na(exposure_start_date), 1, 0),

    # event after exposure (important outcome)
    event_after_exposure =
      ifelse(any_comp_exposed == 1, "Yes", "No"),

    # event only before exposure
    event_only_before_exposure =
      ifelse(any_comp_unexposed == 1 & any_comp_exposed == 0, "Yes", "No"),

    days_complication=as.numeric(difftime(first_event_date, Date_retin_dx, units = "days"))
  ) %>%
    mutate(across(
    c(any_comp_unexposed:any_injection_exposed,any_complication),
    ~ recode(., "0" = "No", "1" = "Yes")
  ))



########### patient lvl


  patient_df <- Final_cohort %>%

  arrange(PatientDurableKey, Date_retin_dx) %>%

  group_by(PatientDurableKey) %>%

  summarise(

    #---------------------------------------
    # DEMOGRAPHICS
    #---------------------------------------

    Age_retin_dx = first(Age_retin_dx),
    Sex = first(Sex),
    Ethnicity = first(Ethnicity),
    FirstRace = first(FirstRace),
    Language = first(Language),
    MaritalStatus = first(MaritalStatus),
    insurance = first(insurance),
    region = first(region),
    ruca = first(ruca),

    #---------------------------------------
    # SOCIAL VULNERABILITY
    #---------------------------------------

    svi_overall_2020 = first(svi_overall_2020),
    svi_overall_2020_cat = first(svi_overall_2020_cat),
    svi_ses_2020 = first(svi_ses_2020),
    svi_house_2020 = first(svi_house_2020),
    svi_minority_2020 = first(svi_minority_2020),
    svi_transport_2020 = first(svi_transport_2020),

    #---------------------------------------
    # SICKLE CELL VARIABLES
    #---------------------------------------

    Genotype = first(Genotype),
    Date_sc_dx = first(Date_SC_dx),
    Age_sc_dx = first(Age_SC_dx),

    #---------------------------------------
    # RETINOPATHY BASELINE
    #---------------------------------------

    index_retin_date = min(Date_retin_dx, na.rm = TRUE),

    # baseline severity at FIRST retinopathy diagnosis
    baseline_severity =
      severity[which.min(Date_retin_dx)],

    baseline_PDR =
      ifelse(
        baseline_severity == "Proliferative",
        1,
        0
      ),

    # any eye with proliferative disease ever
    any_PDR_ever =
      max(severity == "Proliferative", na.rm = TRUE),

    # bilateral disease
    bilateral_eyes =
      n_distinct(Laterality[Laterality %in% c("RT","LT")]),

    #---------------------------------------
    # HYDROXYUREA EXPOSURE
    #---------------------------------------
```

```
    exposed_group = first(exposed_group),

    exposed_at_index =
      ifelse(
        any(exposed_at_index == "Yes", na.rm = TRUE),
        "Yes",
        "No"
      ),

    hydroxyurea_ever =
      ifelse(
        any(!is.na(exposure_start_date)),
        "Yes",
        "No"
      ),

    first_hydroxyurea_date =
      suppressWarnings(
        min(exposure_start_date, na.rm = TRUE)
      ),

    #--------------------------------------------
    # FOLLOW-UP
    #--------------------------------------------

    max_followup_days =
      max(FU_day, na.rm = TRUE),

    max_followup_months =
      max(FU_month, na.rm = TRUE),

    last_encounter =
      max(LastEncounterDate, na.rm = TRUE),

    #--------------------------------------------
    # PROGRESSION
    #--------------------------------------------

    any_progression =
      ifelse(
        any(progressed_flag == "Yes", na.rm = TRUE),
        "Yes",
        "No"
      ),

    progression_date =
      suppressWarnings(
        min(progression_date, na.rm = TRUE)
      ),

    days_to_progression =
      suppressWarnings(
        min(days_to_progression, na.rm = TRUE)
      ),

    n_eyes_progressed =
      sum(progressed_flag == "Yes", na.rm = TRUE),

    #--------------------------------------------
    # MORTALITY
    #--------------------------------------------

    Status = first(Status),
    DeathDate = first(DeathDate),

    .groups = "drop"

  ) %>%

  mutate(

    #--------------------------------------------
    # CLEAN INF VALUES
    #--------------------------------------------

    progression_date =
      as.Date(ifelse(is.infinite(progression_date),
                     NA,
                     progression_date),
              origin = "1970-01-01"),

    days_to_progression =
      ifelse(is.infinite(days_to_progression),
             NA,
             days_to_progression),

    first_hydroxyurea_date =
      as.Date(ifelse(is.infinite(first_hydroxyurea_date),
                     NA,
                     first_hydroxyurea_date),
              origin = "1970-01-01"),

    #--------------------------------------------
    # DERIVED VARIABLES
    #--------------------------------------------

    age_group =
      case_when(
        Age_retin_dx < 18 ~ "<18",
        Age_retin_dx < 40 ~ "18-39",
        Age_retin_dx < 60 ~ "40-59",
        TRUE ~ "60+"
      ),

    progression_years =
      days_to_progression / 365.25
  )

#------------------------------------------------------------
```

```
#-------------------------------------------------------------
# QUICK CHECKS
#=============================================================

# str(patient_df)
#
# table(patient_df$baseline_severity)
#
# table(patient_df$baseline_PDR)
#
# table(patient_df$hydroxyurea_ever)
#
# table(patient_df$any_progression)
#
# summary(patient_df$Age_retin_dx)
#
# summary(patient_df$max_followup_months)



# return(list(
#   va_timepoints=va_timepoints,
#   Final_cohort=Final_cohort,
#   complications=complications,
#   patient_progression_df=patient_progression_df,
#   eye_df=eye_df,
#   patient_df=patient_df
# ))

  # names(eye_df2) <= simple first complication.
  # names(eye_df) <= if using longitudinal complications
  # names(Final_cohort)

  # first_complication_by_exposure <= record of first complication for each period (exposed vs unexposed)

  save(va_timepoints,
       # eye_df,
       patient_df,
       first_complication_by_exposure,eye_df2,
       gee_comp, complication_boxplt,
       file="data/report_data.RData")

}



# testdf <- eye_df2 %>% select(UID,Date_retin_dx,Date_SC_dx,exposed_group,severity,first_npdr_date:progressed_flag,
hydroxyurea_exposed_event,first_severe_date:time_to_severe,FU_month)


 refine_data()

  # va_timepoints=finaldata$va_timepoints
  # eye_df=finaldata$eye_df
  # patient_df=finaldata$patient_df
  # Final_cohort=finaldata$Final_cohort
  # complications=finaldata$complications
   # patient_progression_df=finaldata$patient_progression_df




# names(Final_cohort)

# table(Final_cohort$exposed_group)
```
subset data

```{r}
# Mon Jun 22 14:26:24 2026 ------------------------------

library(tidyverse)
library(bit64)

###Subset analysis
 subset_data <- function() {


    load("data/report_data.RData")
 load("for_subsetdata.RData")

# names(first_complication_by_exposure)

# table(eye_df2$Genotype)


# eye_df2 <- eye_df2 %>%  filter(Genotype %in% c('Hb-SB','Hb-SC')) %>% droplevels()

eye_df2 <- eye_df2 %>% filter(Genotype !='Hb-AS') %>% droplevels()

eye_df2$Genotype <- relevel(eye_df2$Genotype, ref = 'Hb-SS')


va_timepoints <- va_timepoints %>%
  mutate(UID=paste0(PatientDurableKey, "_", Laterality)) %>%
  filter(UID %in% eye_df2$UID ) %>% droplevels()



  patient_df <- patient_df %>%
    filter(PatientDurableKey %in% eye_df2$PatientDurableKey) %>% droplevels()



first_complication_by_exposure <- first_complication_by_exposure %>%
  filter(UID %in% eye_df2$UID )


#-----------------------------------
library(geepack)
```

```r
eye_burden <- eye_burden %>%
  filter(UID %in% eye_df2$UID ) %>% droplevels()

eye_burden$Genotype <- relevel(eye_burden$Genotype, ref = 'Hb-SS')

gee_comp <- geeglm(

  n_total_complications ~
    exposed_group +
    Age_retin_dx +
    Sex +
    Genotype +
    offset(log(FU_years)),

  id = PatientDurableKey,

  family = poisson(link = "log"),

  corstr = "exchangeable",

  data = eye_burden
)

# summary(gee_comp)
#
#
# library(glmmTMB)
#
# model <- glmmTMB(
#   n_total_complications ~ exposed_group + Age_retin_dx + Sex + Genotype + offset(log(FU_years)) +
#     (1 | PatientDurableKey),
#   family = nbinom2,
#   data = eye_burden
# )
#
# summary(model)
#
# summary(gee_comp)



# table(eye_burden$n_total_complications,eye_burden$Genotype)



complications <- complications %>%
  filter(UID %in% eye_df2$UID )%>% droplevels()



comp_counts <-complications %>%
    filter(!is.na(complication_type)) %>%

  group_by(
    UID,
    complication_type,
    hydroxyurea_exposed_event
  ) %>%

  summarise(

    n_complications = n(),

    FU_days = max(Days_complication, na.rm = TRUE),

    FU_years = FU_days / 365.25,

    complication_rate =
      n_complications / FU_years,

    .groups = "drop"
  )


complication_boxplt <- ggplot(
  comp_counts %>% mutate(hydroxyurea_exposed_event=ifelse(hydroxyurea_exposed_event==0,'No','Yes')),

  aes(
    x = hydroxyurea_exposed_event,
    y = n_complications,
    group=hydroxyurea_exposed_event
  )

) +

  geom_boxplot() +

  geom_jitter(
    width = 0.15,
    alpha = 0.4
  ) +

  labs(
    title = "number of complications by hydroxyurea exposure",

    x = NULL,

    y = "Complications count"
  ) +

  theme_bw()+
  facet_wrap(~complication_type)


# complication_boxplt
```

```
  save(va_timepoints,
       # eye_df,
       patient_df,
       first_complication_by_exposure,eye_df2,
       gee_comp, complication_boxplt,
       file="data/report_data_subset.RData")


 }

 subset_data()




 ######################### complication burden



comp_eye_tv <- complications%>%filter(!is.na(complication_type)) %>%

  group_by(UID) %>%

  summarise(

    #-------------------------------------------
    # ANY EVENT (overall)
    #-------------------------------------------

    any_complication = 1,

    #-------------------------------------------
    # UNEXPOSED PERIOD EVENTS
    #-------------------------------------------

    any_comp_unexposed =
      as.integer(any(hydroxyurea_exposed_event == 0)),

    any_vh_unexposed =
      as.integer(any(complication_type == "vitreous_hemorrhage" &
                     hydroxyurea_exposed_event == 0)),

    any_trd_unexposed =
      as.integer(any(complication_type == "tractional_rd" &
                     hydroxyurea_exposed_event == 0)),

    any_prp_unexposed =
      as.integer(any(complication_type == "PRP" &
                     hydroxyurea_exposed_event == 0)),

    any_vitrectomy_unexposed =
      as.integer(any(complication_type == "vitrectomy" &
                     hydroxyurea_exposed_event == 0)),

    any_injection_unexposed =
      as.integer(any(complication_type == "injection" &
                     hydroxyurea_exposed_event == 0)),

    #-------------------------------------------
    # EXPOSED PERIOD EVENTS
    #-------------------------------------------

    any_comp_exposed =
      as.integer(any(hydroxyurea_exposed_event == 1)),

    any_vh_exposed =
      as.integer(any(complication_type == "vitreous_hemorrhage" &
                     hydroxyurea_exposed_event == 1)),

    any_trd_exposed =
      as.integer(any(complication_type == "tractional_rd" &
                     hydroxyurea_exposed_event == 1)),

    any_prp_exposed =
      as.integer(any(complication_type == "PRP" &
                     hydroxyurea_exposed_event == 1)),

    any_vitrectomy_exposed =
      as.integer(any(complication_type == "vitrectomy" &
                     hydroxyurea_exposed_event == 1)),

    any_injection_exposed =
      as.integer(any(complication_type == "injection" &
                     hydroxyurea_exposed_event == 1)),

    #-------------------------------------------
    # COUNTS (VERY IMPORTANT)
    #-------------------------------------------

    n_events_total = n(),

    n_events_unexposed =
      sum(hydroxyurea_exposed_event == 0),
```

```
    n_events_exposed =
      sum(hydroxyurea_exposed_event == 1),

    #-----------------------------------------
    # FIRST EVENT TIMING
    #-----------------------------------------

    first_event_date =
      min(Date_complication, na.rm = TRUE),

    first_event_exposed =
      hydroxyurea_exposed_event[
        which.min(Date_complication)
      ],

    .groups = "drop"
  )


eye_df <- eye_df2 %>%

  left_join(comp_eye_tv,
            by = c("UID")) %>%

  mutate(

    # replace NA → 0 for all flags
    across(
      starts_with("any_"),
      ~ ifelse(is.na(.), 0, .)
    ),

    across(
      starts_with("n_events"),
      ~ ifelse(is.na(.), 0, .)
    )
  )%>%

  mutate(

    # ever exposed (time-varying simplified)
    ever_exposed =
      ifelse(!is.na(exposure_start_date), 1, 0),

    # event after exposure (important outcome)
    event_after_exposure =
      ifelse(any_comp_exposed == 1, "Yes", "No"),

    # event only before exposure
    event_only_before_exposure =
      ifelse(any_comp_unexposed == 1 & any_comp_exposed == 0, "Yes", "No"),

    days_complication=as.numeric(difftime(first_event_date, Date_retin_dx, units = "days"))
  ) %>%
    mutate(across(
    c(any_comp_unexposed:any_injection_exposed,any_complication),
    ~ recode(., "0" = "No", "1" = "Yes")
  ))


```

```{r}
library(gtsummary)
library(flextable)

names(Final_cohort)


table(Final_cohort$exposed_group,Final_cohort$exposed_at_index)


Final_cohort_first_eye <- Final_cohort %>%
  group_by(PatientDurableKey) %>%
  slice_min(Date_retin_dx, n = 1, with_ties = FALSE) %>%
  ungroup()




# patients with both eyes
Final_cohort %>%
  group_by(PatientDurableKey) %>%
  summarise(n_eyes = n()) %>%
  filter(n_eyes == 2) %>%
  nrow()

# patients with one eye in each group
 Final_cohort %>%
  group_by(PatientDurableKey) %>%
  summarise(n_groups = n_distinct(exposed_group)) %>%
  filter(n_groups == 2) %>%
  nrow()

 str(Final_cohort)


Final_cohort_first_eye %>%
  mutate(
         # hydroxyurea_ever = factor(hydroxyurea_ever, levels = c("0", "1"), labels = c("No hydroxyurea", "hydroxyurea ever")),
         hydroxyurea_active_at_index= factor(exposed_at_index, levels = c("0", "1"), labels = c("No" , "Yes"))
         ) %>%
  tbl_summary(by=exposed_group, include=c(Sex,FirstRace,Ethnicity,Language,MultiRacial,Status,#StateOrProvince,
                                          svi_overall_2020,svi_ses_2020,svi_house_2020,svi_minority_2020,svi_transport_2020,ruca,region,

                                          Genotype,severity,Age_retin_dx,Insurance_retin_dx,
                                          hydroxyurea_active_at_index,
```

```
                                          FU_month),
              type = c(where(is.numeric) ~ "continuous"),
              statistic = list(
                all_continuous() ~ "{median} ({p25}, {p75})",
                all_categorical() ~ "{n} ({p}%)"
              ),digits = all_continuous() ~ 2,
              label=list(

                        Age_retin_dx="Age at first sickle cell retinopathy diagnosis (years)",
                        race="Race",
                        FU_month="Follow up (months)",
                        Insurance_retin_dx='Insurance',
                        severity='Retinopathy severity'

                        ),
              missing = "ifany"

                ) %>%
  add_p() %>%
  bold_p(t = 0.05) %>%
  bold_labels() %>%
  add_overall() %>%
  modify_header(label = "**Variable**") %>%
  as_flex_table() %>%
  # flextable::set_caption("Demographic data of eyes.") %>%
  # flextable::add_footer_row(values = "Note: 8010 had no baseline IOP value", colwidths = 5) %>%
  #   flextable::add_footer_row(values = "Note: 'Unknown' under the complication type category are those eyes that had no complications. All other
'Unknown' are true unknowns from the database" , colwidths = 5) %>%
  flextable::theme_vanilla()



va_timepoints_plt <- va_timepoints %>%
    mutate(UID=paste0(PatientDurableKey, "_", Laterality)
        ) %>%
  left_join(Final_cohort %>% select(UID,exposed_group) #,exposed_at_index,Genotype,severity
              , by='UID'
            ) %>% select(UID,exposed_group,baseline_va:va_5y)


va_timepoints_plt_long <- va_timepoints_plt %>%
  pivot_longer(
    cols = c(baseline_va, va_3m, va_6m, va_9m, va_1y, va_2y, va_3y, va_5y),
    names_to = "time",
    values_to = "va"
  )%>%
  mutate(
    time = factor(time,
      levels = c("baseline_va","va_3m","va_6m","va_9m","va_1y","va_2y","va_3y","va_5y"),
      labels = c("Baseline","3m","6m","9m","1y","2y","3y","5y")
    )
  )

summary_df <- va_timepoints_plt_long %>%
  group_by(exposed_group, time) %>%
  summarise(
    mean_va = mean(va, na.rm = TRUE),
    se = sd(va, na.rm = TRUE) / sqrt(sum(!is.na(va))),
    .groups = "drop"
  )

ggplot(summary_df, aes(time, mean_va,
                       color = factor(exposed_group),
                       group = exposed_group)) +
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_va - se, ymax = mean_va + se),
                width = 0.15, size = 0.8) +
  scale_color_manual(
    values = c("#1b9e77", "#d95f02"),
    labels = c("No Hydroxyurea", "Hydroxyurea")
  ) +
  labs(
    x = "Follow-up Time",
    y = "Visual Acuity",
    color = "Hydroxyurea Exposure"
  ) +
  theme_classic(base_size = 14)


ggplot(summary_df, aes(time, mean_va,
                       color = factor(exposed_group),
                       group = exposed_group)) +
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  geom_ribbon(aes(ymin = mean_va - 1.96*se,
                  ymax = mean_va + 1.96*se,
                  fill = factor(exposed_group)),
              alpha = 0.2,
              color = NA) +
  scale_color_manual(values = c("#1b9e77", "#d95f02")) +
  scale_fill_manual(values = c("#1b9e77", "#d95f02")) +
  labs(x = "Follow-up Time", y = "Visual Acuity",
       color = "Hydroxyurea", fill = "Hydroxyurea") +
  theme_classic(base_size = 14)




n_df <- va_timepoints_plt_long %>%
  group_by(time, exposed_group) %>%
  summarise(
    n = sum(!is.na(va)),
    .groups = "drop"
  )


ggplot(va_timepoints_plt_long, aes(x = time, y = va, fill = factor(exposed_group))) +
```

```r
  geom_boxplot(position = position_dodge(width = 0.8), outlier.shape = NA) +
  geom_jitter(position = position_jitterdodge(0.2), alpha = 0.4)+
  geom_text(
    data = n_df,
    aes(label = paste0("n=", n), y = Inf),
    position = position_dodge(width = 0.8),
    vjust = 1.5,
    size = 3.5
  ) +
  scale_fill_manual(
    values = c("#1b9e77", "#d95f02"),
    # labels = c("No Hydroxyurea", "Hydroxyurea")
  ) +
  labs(
    x = "Follow-up Time",
    y = "Visual Acuity",
    fill = "Hydroxyurea"
  ) +
  theme_classic(base_size = 14) +
  coord_cartesian(clip = "off")


complications_reduced <- complication_events %>% filter(!is.na(hydroxyurea_exposed_event))%>%
  mutate(UID = paste0(PatientDurableKey, "_", Laterality))


comp_summary <- complications_reduced %>%
  distinct(UID, complication_type, hydroxyurea_exposed_event) %>%
  count(hydroxyurea_exposed_event, complication_type)


ggplot(comp_summary,
       aes(x = complication_type,
           y = n,
           fill = factor(hydroxyurea_exposed_event))) +
  geom_col(position = position_dodge(width = 0.8)) +

  geom_text(
    aes(label = n),
    position = position_dodge(width = 0.8),
    vjust = -0.4,
    size = 4
  ) +

  scale_fill_manual(
    values = c("#1b9e77", "#d95f02"),
    labels = c("Unexposed", "Hydroxyurea Exposed")
  ) +

  labs(
    x = "Complication Type",
    y = "Number of Eyes",
    fill = "Hydroxyurea Exposure"
  ) +

  theme_classic(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 30, hjust = 1)
  ) +

  expand_limits(y = max(comp_summary$n) * 1.1)



ggplot(comp_summary,
       aes(x = factor(hydroxyurea_exposed_event),
           y = n,
           fill = complication_type)) +
  geom_col(position = "fill") +
  labs(
    x = "Hydroxyurea Exposure",
    y = "Proportion of Complications",
    fill = "Complication"
  ) +
  scale_y_continuous(labels = scales::percent) +
  theme_classic(base_size = 14)



sample_size <- va_timepoints_plt %>% group_by(hydroxyurea_active_at_index) %>% summarise(num=n())

############## plots
ggplot(
  Final_cohort,
  aes(
    x = hydroxyurea_active_at_index,
    y = svi_overall_2020,
    fill = DisplayString_retin_dx2
  )
) +
  geom_violin(width = 1.4) +
  geom_boxplot(width = 0.1, color = "grey", alpha = 0.2) +
  scale_x_discrete(
    labels = function(x) {
      paste0(x, "\n(n=", scales::comma(sample_size$num[match(x, sample_size$hydroxyurea_active_at_index)]), ")")
    }
  )+ylab('SVI: Overall')+xlab('Retinopathy severity')+ggpubr::theme_pubr()


ggplot(
  cohort,
  aes(
    x = retinopathy_severity_ordered,
    y = ClosestA1c,
    fill = retinopathy_severity
  )
) +
  geom_violin(width = 1.4) +
  geom_boxplot(width = 0.1, color = "grey", alpha = 0.2) +
  scale_x_discrete(
```

```
    labels = function(x) {
      paste0(x, "\n(n=", scales::comma(sample_size$num[match(x, sample_size$retinopathy_severity_ordered)]), ")")
    }
  )+ylab('A1c')+xlab('Retinopathy severity')+ggpubr::theme_pubr()


ggplot(
  cohort,
  aes(
    x = retinopathy_severity_ordered,
    y = Baseline_va,
    fill = retinopathy_severity
  )
) +
  geom_violin(width = 1.4) +
  geom_boxplot(width = 0.1, color = "grey", alpha = 0.2) +
  scale_x_discrete(
    labels = function(x) {
      paste0(x, "\n(n=", scales::comma(sample_size$num[match(x, sample_size$retinopathy_severity_ordered)]), ")")
    }
  )+ylab('VA')+xlab('Retinopathy severity')+ggpubr::theme_pubr()
# ggplot(cohort, aes(x = svi_overall_2020)) +
#   geom_histogram( fill = "blue", color = "black") +
#   facet_wrap(~retinopathy_severity) +
#   theme_minimal()









multi_data <- demo_data_filtered %>%
  mutate(
        # severity_rank2= factor(severity_rank,
        #                        levels = c(0,1,2,3,4),
        #                        labels = c("NoComp", "NPDR_Mild", "NPDR_Moderate", "NPDR_Severe", "PDR"),
        #                        ordered = TRUE),
        #  diabetes_type=case_when(
        #    diabetes_type=='DM due to underlying condition' | diabetes_type=='Drug or chemical induced DM' ~'Generic',
        #    TRUE~diabetes_type),
         FirstRace=factor(case_when(
           FirstRace=='American Indian or Alaska Native' | FirstRace=='Native Hawaiian or Other Pacific Islander'  | FirstRace=='Other Race'~'Other
Race',
           TRUE~FirstRace
         ))

         ) %>%
dplyr::select(-c(starts_with('AH'),insurance_status,Has_Anti_hypertensive,severity_rank,Laterality,
            max_fu,seen_by_endo,
            EncounterCountAfterFirstDate,Endo_LastEncounterAge,Endo_LastEncounterDate,LastEncounterAge,Endo_LastEncounterAge,Endo_LastEncounterDate,
            MultiRacial,
            TotalA1cMeasurements,ClosestA1c,closestA1c_date)) %>%
  filter(complete.cases(.))


# Calculate tertile breaks
# tertile_breaks <- quantile(patient_level_data$Years, probs = seq(0, 1, by = 1/3), na.rm = TRUE)
# labels <- c(
#   paste("Low (", round(tertile_breaks[1], 2), "-", round(tertile_breaks[2], 2), ")", sep = ""),
#   paste("Medium (", round(tertile_breaks[2], 2), "-", round(tertile_breaks[3], 2), ")", sep = ""),
#   paste("High (", round(tertile_breaks[3], 2), "-", round(tertile_breaks[4], 2), ")", sep = "")
# )
#   # Categorize IOP into tertiles
# patient_level_data$age <- cut(patient_level_data$Years, breaks = tertile_breaks, include.lowest = TRUE, labels = labels)

age_breaks <- c(19, 30, 45, 65, 80, 116)
age_labels <- c("19-29", "30-44", "45-64", "65-79", "80+")

multi_data$age <- cut(multi_data$Years,
                           breaks = age_breaks,
                           labels = age_labels,
                           right = FALSE,  # so e.g. 30 falls into "30-44"
                           include.lowest = TRUE)




multi_data$SmokingStatus <- relevel(multi_data$SmokingStatus,ref = 'Never')
multi_data$FirstRace <- relevel(multi_data$FirstRace, ref = 'White')
multi_data$CoverageFinancialClass <- relevel(multi_data$CoverageFinancialClass, ref = 'Medicare')
multi_data$ruca <- relevel(multi_data$ruca, ref = 'Urban')

multi_data$age <- factor(multi_data$age)









  convert_to_factors <- function(data, cols) {
    data |>
      mutate(across(all_of(cols), as.factor))
  }

  character_cols <- demo_data_filtered%>% dplyr::select(-PatientDurableKey,-Value) %>% dplyr::select(where(is.character)) %>% colnames()

  # character_cols
  demo_data_filtered <- convert_to_factors(demo_data_filtered, character_cols)
```

```
    #numeric
 numeric_cols_binary <- demo_data_filtered %>%
dplyr::select(-c(PatientDurableKey,Years,svi_overall_2020:Endo_LastEncounterDate,TotalA1cMeasurements,ClosestA1c,severity_rank,max_fu)) %>%
dplyr::select(where(is.numeric)) %>% colnames()
  # Convert specific binary columns in a dataframe
demo_data_filtered <- demo_data_filtered %>%
  mutate(across(all_of(numeric_cols_binary), ~ factor(ifelse(. == 1, "Yes", "No"), levels = c("No", "Yes"))))




transform_to_percentage <- function(x) {
  # Check if the input column is numeric and in the range 0 to 1
  if(is.numeric(x) && all(x >= 0 & x <= 1, na.rm = TRUE)) {
    # Transform the numeric column to factors based on the ranges
    cut(x,
        breaks = c(-Inf, 0.25, 0.5, 0.75, Inf),
        labels = c("0-25%", "25-50%", "50-75%", "75-100%"),
        right = FALSE
    )
  } else {
    stop("The input must be numeric and have values in the range 0 to 1.")
  }
}

cohort$svi_overall_2020_cat  <- transform_to_percentage(cohort$svi_overall_2020)


```

simple exposure
```{r}




complications_reduced <- complication_events %>%
  mutate(UID = paste0(PatientDurableKey, "_", Laterality)) %>%
  left_join(cohort_hyrox_use %>%
              mutate(UID=paste0(PatientDurableKey, "_", Laterality)
                     ) %>% select(-c(PatientDurableKey,Laterality))
             , by='UID'
           ) %>%
  filter(!is.na(complication_type))


comp_summary <- complications_reduced %>%
  distinct(UID, complication_type, hydroxyurea_ever) %>%
  count(hydroxyurea_ever, complication_type)


ggplot(comp_summary,
       aes(x = complication_type,
           y = n,
           fill = factor(hydroxyurea_ever))) +
  geom_col(position = position_dodge(width = 0.8)) +

  geom_text(
    aes(label = n),
    position = position_dodge(width = 0.8),
    vjust = -0.4,
    size = 4
  ) +

  scale_fill_manual(
    values = c("#1b9e77", "#d95f02"),
    labels = c("Unexposed", "Hydroxyurea Exposed")
  ) +

  labs(
    x = "Complication Type",
    y = "Number of Eyes",
    fill = "Hydroxyurea Exposure"
  ) +

  theme_classic(base_size = 14) +
  theme(
    axis.text.x = element_text(angle = 30, hjust = 1)
  ) +

  expand_limits(y = max(comp_summary$n) * 1.1)



ggplot(comp_summary,
       aes(x = factor(hydroxyurea_ever),
           y = n,
           fill = complication_type)) +
  geom_col(position = "fill") +
  labs(
    x = "Hydroxyurea Exposure",
    y = "Proportion of Complications",
    fill = "Complication"
  ) +
  scale_y_continuous(labels = scales::percent) +
  theme_classic(base_size = 14)



sample_size <- va_timepoints_plt %>% group_by(hydroxyurea_active_at_index) %>% summarise(num=n())

############## plots
ggplot(
  Final_cohort,
  aes(
    x = hydroxyurea_active_at_index,
    y = svi_overall_2020,
    fill = DisplayString_retin_dx2
  )
) +
```

```
  geom_violin(width = 1.4) +
  geom_boxplot(width = 0.1, color = "grey", alpha = 0.2) +
  scale_x_discrete(
    labels = function(x) {
      paste0(x, "\n(n=", scales::comma(sample_size$num[match(x, sample_size$hydroxyurea_active_at_index)]), ")")
    }
  )+ylab('SVI: Overall')+xlab('Retinopathy severity')+ggpubr::theme_pubr()


ggplot(
  cohort,
  aes(
    x = retinopathy_severity_ordered,
    y = ClosestA1c,
    fill = retinopathy_severity
  )
) +
  geom_violin(width = 1.4) +
  geom_boxplot(width = 0.1, color = "grey", alpha = 0.2) +
  scale_x_discrete(
    labels = function(x) {
      paste0(x, "\n(n=", scales::comma(sample_size$num[match(x, sample_size$retinopathy_severity_ordered)]), ")")
    }
  )+ylab('A1c')+xlab('Retinopathy severity')+ggpubr::theme_pubr()


ggplot(
  cohort,
  aes(
    x = retinopathy_severity_ordered,
    y = Baseline_va,
    fill = retinopathy_severity
  )
) +
  geom_violin(width = 1.4) +
  geom_boxplot(width = 0.1, color = "grey", alpha = 0.2) +
  scale_x_discrete(
    labels = function(x) {
      paste0(x, "\n(n=", scales::comma(sample_size$num[match(x, sample_size$retinopathy_severity_ordered)]), ")")
    }
  )+ylab('VA')+xlab('Retinopathy severity')+ggpubr::theme_pubr()

```